

2026 年华北五省（市、自治区）及港澳台大学生

计算机应用大赛 移动互联网应用创新

移动互联网应用程序开发

【项目名称】

铁程 tripai——对话式智能火车出行规划

（Web 应用作品）

所在赛区：_____

所在学校：_____

团队名称：_____

团队成员：_____

指导教师：_____

提交日期：_____

2026 年华北五省（市、自治区）及港澳台大学生计算机应用大赛 参赛作品知识产权声明

参赛作品名称：铁程 tripai——对话式智能火车出行规划（下称该作品）。

本小组全体成员，就该作品声明如下：

- 1、 该作品（含提交参赛的 Web 应用、相关文档和介绍视频）是本小组成员的原创研究成果，不存在侵犯任何他人知识产权、涉及泄密或违反中华人民共和国法律法规的情形，且未在规定时间内公开发布。
- 2、 我们确认：该作品知识产权归本小组成员所有，且签署本声明的人员能够代表全体小组成员的意愿。
- 3、 我们承诺：将仅由本小组成员参加大赛答辩。
- 4、 大赛举办方为宣传大赛、推广参赛作品，以及为未来各届大赛的参赛选手提供参考等非盈利目的，可能需要以下列形式使用参赛作品，就此我们确认：
 - （1） ☐ 同意 ☐ 不同意：将参赛作品的相关文档结集出版（含公开发行）；
 - （2） ☐ 同意 ☐ 不同意：在相关网站（包括大赛官网及含优酷等第三方视频平台网站）上传和播放介绍视频或幻灯片；
 - （3） ☐ 同意 ☐ 不同意：在大赛官方网站提供 APP 应用免费下载；
 - （4） 作出上述许可，同样不违反本声明第 1 款的要求。

作品小组全体成员：_____

日期：2026 年__月__日

目录	2
----	---

目录

一、作品概述	3
二、作品可行性分析和目标群体	4
三、作品功能与原型设计	5
四、作品实现、难点及特色分析	6
五、团队介绍和人员分工	9
六、其他	10
七、致谢	11

一、作品概述

“铁程 tripai”是一个面向个人出行者的对话式智能火车出行规划系统。传统查询工具通常要求用户逐项填写车站、日期、坐席和换乘条件，遇到直达无票时，用户还要自行拆分区间、比较中转站并逐车核实余票。本作品将这一过程交给 AI Agent：用户用一句自然语言描述“何时、几人、从哪里到哪去、坐什么席别、偏好什么时间”，系统自动完成车站消歧、本地线路枚举、12306 实时余票与票价核实、补票变体分析和正式报告生成。

作品的核心价值是把“本地全路网静态推理”和“联网实时票额确认”分成两层。本地层保存车站、车次、经停和历史票价，可在离线状态下快速枚举直达和最多两次换乘方案；联网层只对推荐候选调用 12306 接口，减少无效请求并保证最终建议可购买、可执行。系统同时保留手动查票模式，满足需求明确用户的精确筛选和快速直查要求。

二、作品可行性分析和目标群体

（1）可行性分析

技术可行性

后端采用 Python 3 和 FastAPI, 数据处理使用 SQLite, 前端使用原生 HTML/CSS/JavaScript, 服务端事件 (SSE) 承担流式输出。12306 公开查询接口可核实余票和票价, OpenAI 兼容接口可接入不同大模型。当前本地库包含 3404 个车站、16076 趟车次、162943 条经停记录和约 279 万条历史票价记录, 具备支撑全路网方案枚举的数据基础。

操作可行性

用户无需注册登录。首次使用只需配置 API-KEY、模型名称和接口地址; 日常使用可双击启动脚本, 浏览器自动打开本地服务。AI 模式适合自然语言提问, 手动模式提供表单筛选, 两类用户均能低门槛上手。

经济可行性

系统主体可运行在个人电脑或一台低配置服务器上, SQLite 文件部署简单; 模型服务由用户选择已有 OpenAI 兼容账号承担, 12306 核实接口按需调用, 不引入额外数据库许可成本。

（2）目标群体

核心用户是有火车出行需求的旅客, 尤其是在直达无票时寻找中转方案的学生、上班族、商务出行者和跨城通勤人群。辅助用户包括需要筛选特定时间窗、坐席或车型的精确查询用户, 以及需要保留规划过程、比较多个候选方案的团队出行组织者。

三、作品功能与原型设计

(1) 功能概述

功能	说明
AI 对话规划	用户用自然语言提出需求；系统展示思考流和工具调用轨迹，流式生成正式规划报告与方案总表，支持连续追问和多轮调整。
本地方案枚举	基于本地车站、车次、经停数据枚举直达、一次换乘和两次换乘方案，执行换乘间隔、当日到达、时间窗、车型等硬约束。
实时余票核实	对推荐方案调用 12306 接口确认余票和票价，识别不可购区间；无票时继续分析前后多站、买短补长等变体。
对话详情	全屏平铺会话中的全部结构化方案，包括无票方案；支持逐条或批量联网核实，便于用户比较全量候选。
手动查票	不经 AI 直接连接查票引擎，支持站名模糊匹配、日期、坐席、换乘次数、时间范围、车型、排序和返回数量筛选。
会话持久化	每个会话保存为 JSON 文件，包含消息、思考链、工具轨迹和方案数据；重启后可恢复，历史会话可完整回放。
数据采集维护	提供车站、车次经停、票价采集和异常清理脚本，支持断点续爬、并发限速、单趟补采和状态统计。

(2) 原型设计

主界面采用“模式选择—具体视图”的两级结构。模式选择页突出 AI 对话与手动查票两条路径；AI 视图左侧为会话列表，中部为思考流、正式回答和方案总表；对话详情页按卡片平铺候选方案，并保留联网核实按钮；手动查票页左侧为筛选表单，右侧为结果列表。整体使用人大红与米白纸感配色，突出正式、稳定和易读的出行助手形象。

交互设计遵循三项原则：第一，AI 结论必须能追溯到车次、区间、票价和余票状态；第二，本地枚举结果与联网核实状态分离展示，未核实数据不能被误认为可购票；第三，长任务提供查询中、检查中、思考秒数等反馈，并允许用户停止生成。

四、作品实现、难点及特色分析

(1) 作品实现及难点

1. 总体架构

系统分为采集层、数据层、规划引擎、AI Agent 层、服务接口层和前端表现层。`collect` 包负责静态数据采集；`data/railway.db` 保存车站、车次和经停，`data/prices.db` 保存历史票价；`app/planner.py` 执行纯本地规划；`app/tools.py` 定义 Agent 可调用工具；`app/online.py` 封装 12306 实时核实；`app/server.py` 提供 REST 和 SSE 接口；`static` 目录实现响应式页面。

2. 开发周期设计过程

周期	主要工作与产出
需求分析	梳理直达查询、跨天列车、多次换乘、无票补救、实时票额、多轮对话和会话回放等场景，确定“AI 规划”与“手动直查”双模式。
数据设计	设计车站表、车次表、经停表、站车索引和历史票价表；为站对查询、车次编号匹配、改号车别名和换乘枚举建立索引。
架构设计	确定本地枚举与联网核实分离、SQLite 轻量部署、SSE 流式响应、JSON 会话落盘和采集脚本独立运行的技术路线。
详细设计	定义换乘图搜索约束、评分公式、工具协议、方案数据结构、票额状态、异常提示和前端卡片层级。
编码实现	实现采集器、规划引擎、Agent 工具循环、12306 核实、会话管理、前端渲染与批量核实接口。
测试与修正	针对大城市/小站、直达无票、一次换乘、两次换乘、跨零点、非每日车、改号车和接口反爬场景做功能验证。
部署运维	提供启动脚本、采集脚本、日志目录、断点续爬和状态统计；支持按字头小规模试采和全量重建。

3. 关键实现

规划引擎在 SQL 阶段过滤可乘区间，再进行链式换乘搜索。换乘衔接不少于 20 分钟，后续程车不经过原出发站，非末段车不经过目的地，避免“看似换乘、实则可直达”的伪方案。链式搜索每层保留 40 条路径，并根据累计历时截断，防止枢纽站组合爆炸。未指定换乘次数时，系统自动从直达向一转、两转扩展，直到候选数满足配额。

综合评分由用时、出发时间、换乘代价和等待时间组成，直达不计算等待惩罚。AI 工具从约 $N \times 20$ 个去重候选中选择前 N 项供模型分析，同时把前 $2N$ 项送入前端详情页，兼顾回答聚焦与全量可比。

实时核实层将 12306 返回的坐席、余票、票价映射为统一状态，按 `train_no` 兜底改号车，并对非每日车尝试多个日期偏移。多区间请求会合并查询，减少接口压力。前端使用 `marked` 渲染 Markdown，并用 `DOMPurify` 净化输出。

4. 主要难点

- 全网换乘组合规模大：通过层间截断、累计历时限制、城市过滤和配额控制保证响应速度。
- 12306 数据语义复杂：对电报码、显示车次、内部 `train_no`、多日期开行和部分区间票额建立统一解析规则。
- AI 结果需要稳定：模型只负责理解需求、组织工具调用和解释结果，路线计算与票额判定由确定性代码完成。
- 长任务体验：SSE 分段输出思考流与答案，工具轨迹可展开，用户能观察进度并主动停止。

（2）特色分析

- 原生 Agent 闭环：不是简单让模型推荐车次，而是模型连续调用本地规划工具、在线核实工具和补票变体工具，结论由数据和代码支撑。
- “全量+推荐”双视图：正式报告只展示可购或明确标记的方案，对话详情保留全部候选，避免用户丢失被 AI 摘要过滤的信息。
- 补票变体深挖：直达无票时自动分析前后站、买短乘长和分段组合，并明确区分“可购”“无票/受限”“未核实”。
- 轻量可维护：SQLite 文件库、脚本化采集、断点续爬和日志留存使系统无需复杂数据库服务也能长期维护。

（3）测试账号与配置说明

本软件不设置注册用户名和密码，无需测试账号。首次进入需在本机配置用户自己的 API-KEY、模型名称和 OpenAI 兼容接口地址；该配置保存在项目目录的 `.env` 文件中，不随作品源码提交。若评审环境没有模型服务，可先使用手动查票模式体验本地方案枚举，联网核实功能另需可访问 12306 的网络。

（4）操作步骤及说明

步骤一：启动系统

首次安装依赖后，双击项目根目录的启动铁程 `.bat`；或在命令行进入项目目录并执行 `.venv\Scripts\python.exe -m app.server`。浏览器访问 `http://127.0.0.1:8600`。

步骤二：配置模型

在配置网页填写 API-KEY、模型名称和接口地址，点击“验证并进入”。配置只保存在本机 `.env`；后续可在顶栏“设置”修改。

步骤三：AI 对话规划

选择“AI 对话模式”，在输入框描述需求，例如“下周六 3 个人从北京去上海，尽量上午出发，二等座”。发送后系统显示思考流、工具调用和正式报告；可在方案总表中查看评分、时刻、票价、余票状态，并继续追问或改期。

步骤四：查看对话详情

点击顶栏“对话详情”查看本次会话全部结构化方案。输入前 N 条数量后点击“批量核实”，或对单条方案点击“联网核实”。核实完成后，状态会更新为可购、无票/受限或其他提示。

步骤五：手动查票

点击印章图标返回模式选择，进入“手动查票模式”。填写或选择出发站、到达站、出行日期、坐席、换乘次数、时间范围、车型和排序；点击“查询方案”查看本地候选。结果右侧提供“联网核实”，可获得 12306 实时余票和票价。

步骤六：会话与数据维护

AI 视图左侧可新建会话；右键会话可重命名或删除。切换历史会话可回放思考链和方案。数据维护可在项目根目录执行 `python -m collect.collector --stats`、`python -m collect.price_collector --stats` 查看状态，使用 `--resume` 续爬；全量采集建议按 README 中的车站、车次、票价顺序执行。

（5）测试设计与质量保障

测试覆盖五类路径：车站消歧测试（北京、北京南、上海虹桥等同城多站）、约束测试（预售期、跨零点、换乘间隔）、枚举测试（直达、一转、两转及无解场景）、接口测试（配置校验、会话创建、SSE 中断、批量核实）和数据测试（改号车别名、非每日车、票价缺失回落）。前端重点验证 Markdown 净化、长列表滚动、状态标签颜色和会话回放一致性。所有网络失败均返回可读错误，不把历史参考价误标为实时票价。

五、团队介绍和人员分工

成员/岗位	分工
队长	需求分析、总体架构、AI Agent 设计、规划引擎联调、文档统稿。
成员 A	数据模型设计、采集器开发、断点续爬与数据清理、索引优化。
成员 B	12306 接口封装、票额解析、多日期兜底、批量核实与异常处理。
成员 C	前端界面、SSE 渲染、方案卡片、对话详情、交互状态与可用性测试。
指导教师	总体方案指导、技术路线评审与作品完善建议。

注：提交前请按参赛报名信息填写成员姓名、学校、学院和指导教师。

六、其他

本作品使用的数据来自公开查询接口采集与整理，仅用于出行研究和个人规划；实时余票和票价仍以 12306 及车站官方销售结果为准。系统不代用户下单，不存储支付信息，不绕过身份认证或风控规则。采集器内置等待和重试机制，使用时应控制频率，遵守相关网站服务条款。

若部署到公网，建议增加反向代理、HTTPS、访问控制和模型密钥隔离；当前默认版本面向个人本机使用，因此未内置多用户账号体系。

七、致谢

感谢指导教师在需求收敛、系统架构和答辩表达上的指导；感谢团队成员在数据采集、引擎调试和界面打磨中的协作；也感谢开源社区和学校提供的实验环境。团队后续将继续优化换乘体验、数据更新效率和模型解释能力，使作品更贴近真实出行决策。